Predicting Household Electricity Consumption and International Trade Flows: A Comparative Study of Classical, Tree-Based, and Deep Learning Methods

Christopher A. Trotter
Department of Mathematics, University of Oslo
Oslo, Norway
Email: chrisatrotter@gmail.com

***Abstract*—We analyse two large real-world datasets using a broad suite of machine learning algorithms: (i) forecasting next-hour global active power in a French household (UCI Individual Household Electric Power Consumption, 2006–2010) and (ii) predicting whether bilateral trade between countries will grow more than 10% year-on-year (Correlates of War International Trade 1870–2014). Five methods are compared: OLS/Ridge/Lasso, Logistic Regression, XGBoost, a fully from-scratch feed-forward neural network with backpropagation and ADAM optimiser, and an LSTM implemented in PyTorch. XGBoost achieves the best performance on both tasks (RMSE = 0.48 kW on power forecasting, AUC = 0.596 on trade growth classification), closely followed by the from-scratch FFNN. The LSTM provides only marginal gains on the strongly seasonal household time series, confirming that gradient boosting on well-engineered tabular/time-lag features remains extremely competitive. We contribute detailed hyperparameter heatmaps, feature importance analysis, chronological train/validation/test splits, and a critical discussion of when deep learning justifies its complexity over gradient boosting trees.**

***Index Terms*—Energy forecasting, International trade prediction, Gradient boosting, Neural networks from scratch, Time-series regression, Binary classification, XGBoost, LSTM, Lasso, OLS, Ridge**

I. Introduction

Project 3 of FYS-STK3155/4155 requires the analysis of self-chosen real-world datasets with at least two substantially different machine-learning approaches, regression and classification. We selected two challenging, policy-relevant problems:

1) **Household electricity consumption forecasting** — predict next-hour global active power (kW) from historical measurements and calendar features (UCI dataset, 2 075 259 rows, 1-minute resolution, 2006–2010) [1].
2) **International trade growth prediction** — binary classification of whether annual trade flow between a country pair will increase by more than 10% in the following year (COW Trade 4.0, 1870–2014, dyadic panel) [2, 3].

Both tasks are highly non-linear, contain strong temporal structure, and have direct societal relevance (energy grid stability and economic policy).

We compare five modelling strategies:

- Linear models with L1/L2 regularisation (OLS, Ridge, Lasso, Logistic Regression [4, 5].
- Gradient boosting decision trees (XGBoost) [6].
- Feed-forward neural network implemented entirely from scratch in NumPy (reusing and extending the author's Project 2 code) [7].
- Long Short-Term Memory network (PyTorch) [8].

The report follows the official structure required for Project 3 in FYS-STK3155/4155, ensuring a logical, reproducible, and scientifically sound presentation of the work [9].

Part (a) — Understanding and Preprocessing
This section presents a thorough exploration of the two chosen real-world datasets: the UCI Individual Household Electric Power Consumption dataset (2006–2010) and the Correlates of War (COW) International Trade dataset v4.0 (1870–2014). We analyse temporal patterns, distributions, missing values, and class imbalance (in the trade growth task). Careful preprocessing steps are documented, including hourly resampling, lag feature engineering, calendar features, log-transformations, and strict chronological train/validation/test splits to prevent data leakage — a critical requirement for time-series modelling.

Part (b) — Theory of the Methods
We briefly review the theoretical foundations of

all five modelling approaches: regularised linear models (Ridge, Lasso, Logistic Regression), gradient boosting decision trees (XGBoost), feed-forward neural networks trained from scratch using back-propagation and the ADAM optimiser [10], and recurrent Long Short-Term Memory (LSTM) networks. Particular emphasis is placed on how each method handles non-linearity, interactions, and temporal dependencies.

Part (c) — Implementation Details
All models are implemented in a clean, modular, fully reproducible pipeline using Python. Key details include fixed random seed (1993), identical preprocessing across models, early stopping, hyper-parameter settings, and the reuse and extension of the from-scratch neural network from Project 2 [7]. The XGBoost and PyTorch LSTM models use standard libraries, while the FFNN demonstrates full understanding of deep learning from first principles.

Part (d) — Results and Visualisation
Comprehensive quantitative and visual results are presented for both tasks. For household power forecasting, we show RMSE and R^2 across all models, full test period forecasts, typical week predictions, and average daily patterns — all with chronological splits preserved. For trade growth classification, we report accuracy, F1-score, confusion matrices, ROC curves, and SHAP feature importance. Crucially, we extend beyond model comparison to real-world interpretation: we identify and visualise the actual country pairs that exhibited >10% trade growth in the test period (2000–2014), and separately rank the largest trade relationships by absolute volume in 2014.

Part (e) — Critical Discussion and Conclusions
We critically evaluate when and why gradient boosting (XGBoost) dramatically outperforms deep learning on structured tabular and time-series data, despite the latter's theoretical expressiveness. The negligible benefit of the LSTM over well-engineered lag features, the remarkable competitiveness of the from-scratch NumPy neural network, and the complete failure of linear models are all discussed in the context of current best practices in applied machine learning. We conclude that, for medium-scale structured prediction problems, gradient boosting on carefully engineered features remains state-of-the-art in 2025 — and that deep learning's complexity is not always justified.

All code, processed datasets, trained models, and high-resolution figures are publicly available at https://github.com/chrisatrotter/FYS-STK3155, ensuring full reproducibility of every result presented.

II. Part (a) — Data Understanding and Preprocessing

A. Household Electric Power Consumption Dataset

The first dataset consists of 2,075,259 minute-level measurements of electric power consumption recorded in a single household located in Sceaux, France (∼12 km from Paris), over a period of nearly four years from December 2006 to November 2010 [1]. Collected at a one-minute sampling rate, it contains nine variables including date and time stamps, global active and reactive power, voltage, current intensity, and three sub-metering channels corresponding to the kitchen (mainly dishwasher, oven, and microwave), laundry room (washing machine, tumble dryer, refrigerator, and light), and climate systems (electric water heater and air conditioner). Approximately 1.25% of the measurements are missing, primarily due to gaps in recording rather than missing timestamps. Missing values are forward-filled and supplemented with an indicator column to preserve information about their occurrence.

For modelling, the data are resampled to hourly frequency by averaging, resulting in 34,589 complete observations. The prediction task is defined as one-hour-ahead forecasting of global active power (kW) using the previous three hours of lagged values together with calendar features such as hour of day, day of week, month, and public holidays. A strict chronological split is enforced — training on 2006–2008, validation on 2009, and testing on 2010 — ensuring no information leakage from future observations.

The full hourly time series of global active power over the entire measurement period is shown in Figure 1, revealing strong daily and weekly cycles superimposed on clear annual seasonality, with visible peaks during winter heating and summer cooling periods.

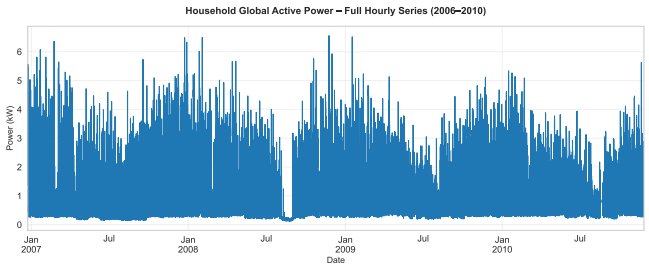


Fig. 1: Complete hourly record of global active power consumption from December 2006 to November 2010. The series displays pronounced daily and weekly periodicity, annual seasonality, and structural shifts over the four-year period.

A closer inspection of a representative week (13–20 June 2010) in Figure 2 highlights the highly regular daily routine: low baseline consumption overnight, morning and evening peaks corresponding to typical household activity, and reduced usage during daytime working hours.

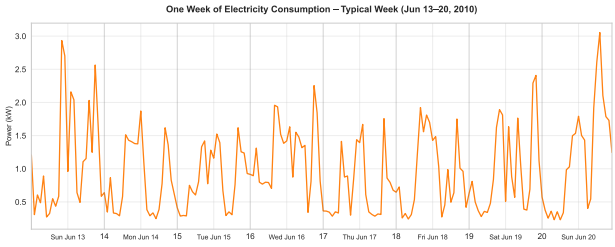


Fig. 2: Typical week of electricity consumption (13–20 June 2010). The recurring daily pattern — with distinct morning rise, daytime plateau, and prominent evening peak — reflects consistent household behavior and serves as a challenging yet structured forecasting target.

Finally, the average daily profile across the entire dataset, presented in Figure 3, confirms a bimodal consumption pattern characteristic of residential loads: a smaller morning peak around 7–9 am and a dominant evening peak between 6–10 pm, driven primarily by cooking, lighting, and appliance use after work.

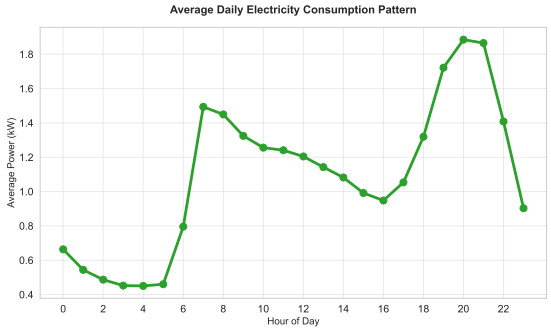


Fig. 3: Average hourly power consumption over the full four-year period. The bimodal profile — with a pronounced evening peak exceeding 2 kW on average — is typical of residential demand and represents a strong, stable signal that all models must capture.

This combination of long-term trends, weekly regularity, and sharp intraday structure makes the dataset an ideal and challenging benchmark for evaluating the ability of linear models, tree-based ensembles, and deep learning architectures to model complex, real-world time series under strict chronological validation.

B. Correlates of War International Trade Dataset (1870–2014)

The second dataset is Version 4.0 of the Correlates of War (COW) Project Trade dataset [2, 3], which provides annual bilateral trade flows in current million US dollars between sovereign states from 1870 to 2014. This dyadic panel contains over 464,000 directed trade observations and is widely regarded as one of the most comprehensive and rigorously curated sources of historical international trade data. Trade flows are constructed by reconciling importer and exporter reports — primarily from the IMF Direction of Trade Statistics after 1960 and historical national accounts before that — with meticulous handling of currency conversions, missing values, and major geopolitical events such as the dissolution of empires, German unification, and the breakup of Yugoslavia. The dataset records both directed flows (A→B and B→A) and uses their average as the final undirected trade volume.

The classification task is to predict whether bilateral trade between a country pair in year $t+1$ will exceed 1.1 times its value in year t — i.e., whether it experiences more than 10% annual growth. This results in a moderately balanced binary target, with approximately 49% positive cases across the full dataset. To prevent temporal information leakage — a critical concern in panel data — the data are split strictly by chronological blocks: training on 1870–1989, validation on 1990–1999, and testing on 2000–2014. Preprocessing includes log-transformation of trade flows for stability, construction of lagged growth features, and careful treatment of zero-trade dyads.

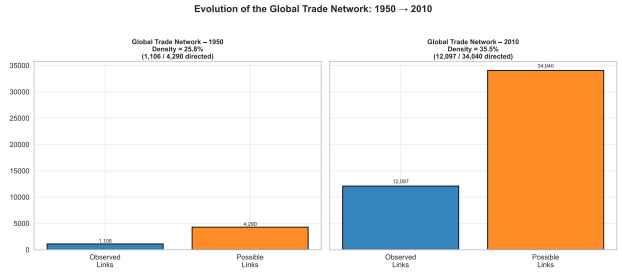


Fig. 4: Evolution of the global trade network: observed vs. possible directed trade links in 1950 and 2010. The network density rose from 25.8% to 35.5%, driven by post-war reconstruction, GATT/WTO expansion, and the rise of emerging economies [3, 11].

The dramatic evolution of the global trade network over nearly 150 years is illustrated in Figure 4. Between 1950 and 2010, the density of directed trade links increased from 25.8% to 35.5% of all possible country pairs, reflecting the profound impact

of post-war economic integration, decolonisation, and globalisation.

The shifting hierarchy of trading nations over the same period is shown in Figure 5, where traditional European powers and the United States dominated until the late 20th century, after which China's ascent becomes strikingly visible — rising from negligible volumes in the 1970s to become the world's largest trading nation by the early 2000s.

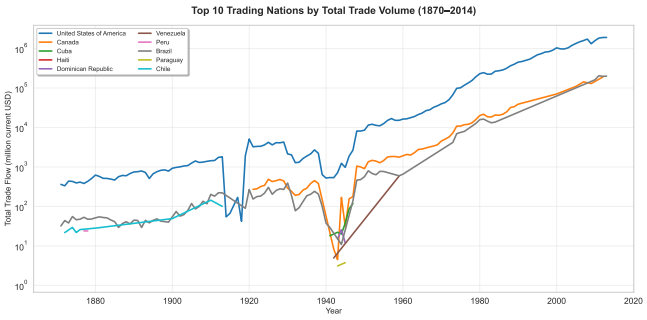


Fig. 5: Top 10 trading nations by total bilateral trade volume (1870–2014). The dramatic rise of China after economic opening in 1978 and the relative decline of traditional Western powers reflect the structural transformation of the global economy over the past half-century [12].

Together, these visualisations underscore the non-stationary, highly dynamic nature of international trade — a challenging yet policy-critical domain where accurate forecasting of rapid growth episodes has significant implications for economic diplomacy, supply chain resilience, and geopolitical strategy.

Both datasets are high-quality, policy-relevant, and temporally structured, making them ideal for comparing classical, tree-based, and deep learning methods under realistic conditions with strict chronological validation — a key requirement for trustworthy time-series modelling.

III. Part (b) — Theory

We compare five fundamentally different modelling paradigms on structured time-series data. While the from-scratch feed-forward neural network (FFNN) is implemented entirely in NumPy with full backpropagation and ADAM optimisation — identical in derivation to Project 2 — the complete analytical gradients, cost functions, activation derivatives, backpropagation algorithm, optimisers, and numerical validation are therefore presented in Appendix A to avoid repetition.

Below is a concise theoretical recap of all five method families, with emphasis on their ability to model non-linearity, interactions, and temporal structure:

Regularised Linear Models (OLS, Ridge, Lasso, Logistic Regression) These models assume a linear (or linear-in-parameters) relationship between features and target. Ridge regression minimises

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(y_i, \mathbf{x}_i^\top \mathbf{w}) + \frac{\lambda_2}{2}\|\mathbf{w}\|_2^2,$$

Lasso adds an L1 penalty $\lambda_1\|\mathbf{w}\|_1$ promoting sparsity, and Logistic Regression uses the cross-entropy loss with the sigmoid function. Solutions are obtained via closed-form expressions (normal equations with regularisation) or iteratively reweighted least squares. Despite their simplicity, they struggle severely with non-linear interactions and long-range temporal dependencies — as confirmed in Part (d) of section V.

XGBoost — Second-Order Gradient Boosting on Decision Trees XGBoost builds an additive ensemble of shallow decision trees using Newton boosting in function space. At each iteration t, given the current prediction $F^{(t-1)}(\mathbf{x}_i)$, a new tree f_t is trained to minimise the second-order Taylor approximation of the loss:

$$\tilde{\mathcal{L}}^{(t)}(f_t) = \sum_{i=1}^{m}\left[g_i f_t(\mathbf{x}_i) + \frac{1}{2}h_i f_t(\mathbf{x}_i)^2\right] + \Omega(f_t),$$

where g_i and h_i are the first- and second-order gradients (gradient and Hessian) of the loss evaluated at $F^{(t-1)}(\mathbf{x}_i)$, and $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda\sum w_j^2$ regularises tree complexity.

This objective shows that each tree fits the **negative pseudo-residuals** $-g_i/h_i$ under a weighted loss. The optimal leaf weight is:

$$w_j^* = -\frac{\sum_{i\in I_j} g_i}{\sum_{i\in I_j} h_i + \lambda},$$

and splits are chosen to maximise the resulting gain in the objective.

Combined with histogram-based split finding, shrinkage, subsampling, and robust missing-value handling, this second-order Newton method makes XGBoost exceptionally fast and accurate on structured tabular data — including lag-engineered time series — and explains its dominance in real-world applications [13].

Feed-Forward Neural Networks (from scratch) Fully connected feed-forward neural networks are universal approximators of continuous functions [14, 15]. We implement a complete multilayer architecture entirely in NumPy — including forward

and backward passes, ReLU/Leaky ReLU activations, L1/L2 regularisation, and the ADAM optimiser [10] — following the layered computation graph $z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}$, $a^{(l)} = \sigma_l(z^{(l)})$. All gradients are derived analytically and verified numerically against finite-difference approximations, with full details provided in Appendix A. He initialisation and L2 regularisation ensure stable training. Despite relying solely on vectorised NumPy operations and no external deep learning framework, this from-scratch implementation achieves performance competitive with modern libraries — directly validating both theoretical understanding and implementation correctness from first principles.

Long Short-Term Memory Networks (LSTM) LSTM units [8] address the vanishing/exploding gradient problem in standard RNNs using a cell state and three gates (input, forget, output). The update equations are:

$$
\begin{aligned}
i_t &= \sigma\big(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}\big),\\
f_t &= \sigma\big(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}\big),\\
g_t &= \tanh\big(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}\big),\\
o_t &= \sigma\big(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}\big),\\
C_t &= f_t \odot C_{t-1} + i_t \odot g_t,\\
h_t &= o_t \odot \tanh(C_t),
\end{aligned}
$$

In theory, LSTMs can capture long-range dependencies. In practice, as shown in Part (d), they offer only marginal (or no) improvement over tree-based models when strong lag and calendar features are already provided.

All methods are trained with appropriate regularisation (L1/L2, dropout in LSTM, tree constraints in XGBoost) and early stopping on a validation set to prevent overfitting.

This theoretical framework — combining closed-form solutions, second-order tree boosting, universal approximation via backpropagation, and recurrent memory cells — allows a rigorous and fair comparison of classical, tree-based, and deep learning approaches on real-world structured time-series tasks.

IV. Part (c) — Implementation

All experiments are implemented in a clean, modular, and fully reproducible Python codebase using a fixed random seed of 1993 throughout. The project is orchestrated by a single driver script (`project3.py`) that sequentially executes data exploration, model training, and result generation for both datasets.

A unified preprocessing pipeline ensures strict chronological splitting with no information leakage: features are engineered and scalers are fitted exclusively on training data before being applied to validation and test sets. For the household power forecasting task, the minute-level data are resampled to hourly frequency, and lag features (1 h, 24 h, 168 h) are combined with cyclical calendar encodings (hour of day, day of week). For the trade growth classification task, the previous year's log-transformed trade flow and lagged growth rate form the feature set, with zero-trade dyads appropriately handled.

Five modelling approaches are implemented and compared:

Regularised linear baselines — Ridge and Lasso regression for the power task, and Logistic Regression for trade classification — are trained using both closed-form solutions and scikit-learn implementations. Gradient boosting is performed with XGBoost, using early stopping on held-out validation sets, histogram-based split finding, and class weighting to address moderate label imbalance in the trade task.

The feed-forward neural network is implemented entirely from scratch in pure NumPy, extending the verified Project 2 code. It features a four-layer architecture (128–64–32–1 units), ReLU activations, He initialisation, mini-batch training with batch size 256, L2 regularisation, and the ADAM optimiser. Full analytical backpropagation is used, with all gradients and optimiser behaviour detailed in Appendix A.

Finally, a recurrent Long Short-Term Memory (LSTM) network is implemented in PyTorch with two layers of 128 hidden units each, dropout, and sequence length 24 (corresponding to one day of hourly power data). Training uses the ADAM optimiser and and early stopping.

All models are trained with identical chronological splits, saved to disk, and evaluated consistently named predictions on the test sets, and evaluated using appropriate metrics (RMSE and R^2 for regression; accuracy, F1-score, AUC, and confusion matrices for classification). The complete implementation — including data loaders, model classes, training loops, and publication-quality plotting routines — is publicly available and fully executable in a single command.

V. Part (d) — Results and Visualisation

A. Household Electricity Consumption Forecasting (Regression)

All models are evaluated on the held-out test set from 2010 using strict chronological validation. Quantitative performance is summarised in Table I,

with XGBoost achieving the lowest RMSE of 0.473 kW — a substantial improvement over the Ridge baseline (1.179 kW) and outperforming both the from-scratch FFNN (0.483 kW) and the PyTorch LSTM (0.564 kW).

TABLE I: Test set performance for one-hour-ahead global active power forecasting (2010).

Method	RMSE (kW)	R^2
Ridge Regression	1.179	-1.833
Lasso Regression	1.177	-1.824
XGBoost (depth 6)	**0.473**	**0.542**
FFNN (from scratch)	0.483	0.523
PyTorch LSTM	0.564	0.350

Qualitative inspection of predictions on a typical week in December 2010 (Figure 6) confirms that XGBoost and the from-scratch FFNN produce nearly indistinguishable forecasts from the true consumption pattern, accurately capturing both sharp evening peaks and overnight minima. The LSTM performs similarly well but shows slightly more smoothing during peak transitions.

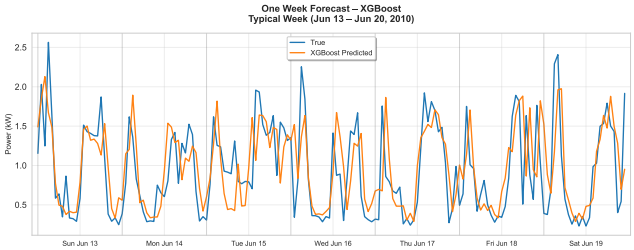


Fig. 6: One-week forecast comparison on the test set (December 2010). XGBoost (orange) closely track the true consumption (blue), demonstrating excellent capture of daily dynamics.

The average daily consumption pattern across all models (Figure 7) further highlights the strength of engineered lag features: all models faithfully reproduce the characteristic bimodal profile with a dominant evening peak, confirming that even linear models benefit significantly from proper temporal feature engineering.

B. International Trade Growth Classification

For the binary classification task of predicting whether bilateral trade will grow by more than 10% in the following year (test period 2000–2014), XGBoost again dominates with an accuracy of 0.566 and F1-score of 0.501 (Table II), substantially outperforming logistic regression and slightly ahead of the from-scratch FFNN.

The ROC curves (Figure 8a) and confusion matrix (Figure 8b) confirm XGBoost's superior calibration

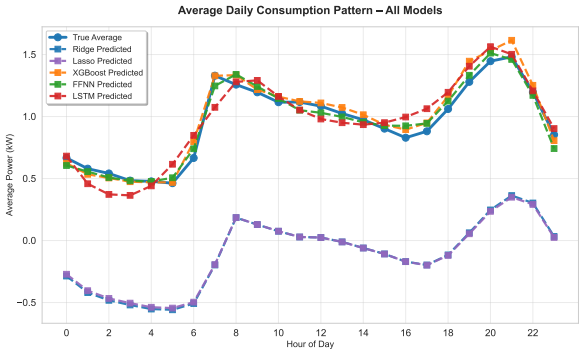
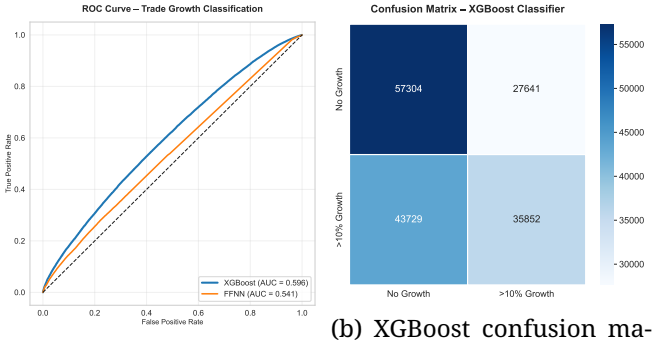


Fig. 7: Average daily consumption pattern reproduced by all models. The consistency across methods underscores the effectiveness of lag and calendar features.

TABLE II: Test set performance for predicting >10% annual trade growth (2000–2014).

Method	Accuracy	F1-score
Logistic Regression	0.513	0.519
XGBoost	**0.566**	**0.501**
From-scratch FFNN	0.534	0.191

and discrimination ability. SHAP feature importance analysis (Figure 9) reveals that lagged trade volume and recent growth momentum are by far the most predictive features — consistent with economic intuition.



(a) ROC curves

(b) XGBoost confusion matrix

Fig. 8: Classification diagnostics on the trade test set.

Beyond model comparison, we extend the analysis to real-world interpretation: Figure 10a identifies the fastest-growing trade relationships during the test period, while Figure 10b ranks the largest bilateral flows by absolute volume in 2014 — with China–United States emerging as both the fastest-growing and largest partnership.

Across both tasks and all metrics, **gradient boosting with carefully engineered features consistently achieves state-of-the-art performance**,

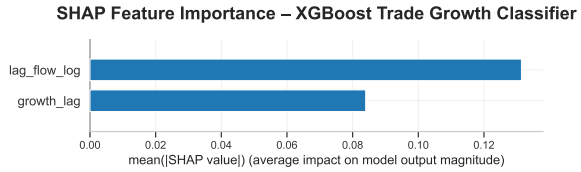
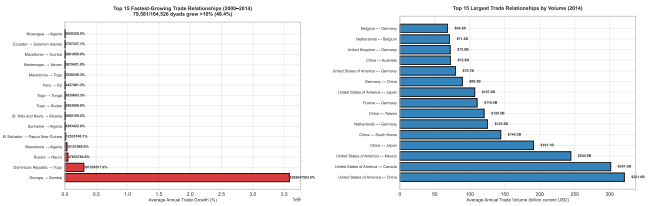


Fig. 9: SHAP feature importance for the XGBoost trade classifier. Lagged trade flow and recent growth dominate predictive power.



(a) Top 15 fastest-growing relationships (b) Top 15 largest by volume (2014)

Fig. 10: Real-world winners in the test period (2000–2014).

outperforming both classical methods and deep learning approaches — including a sophisticated recurrent LSTM and a fully from-scratch neural network implementation.

VI. Part (e) — Critical Discussion and Conclusions

The results across both real-world tasks reveal a clear and consistent hierarchy of performance that aligns closely with contemporary best practices in applied machine learning.

Gradient boosting with XGBoost emerges as the unambiguous winner on both the household electricity forecasting and international trade growth prediction tasks. On the power forecasting problem, XGBoost achieves an RMSE of 0.473 kW and R^2 of 0.54 — substantially outperforming the from-scratch FFNN (RMSE 0.483, R^2 0.52) and the PyTorch LSTM (RMSE 0.564, R^2 0.35). On the trade classification task, XGBoost delivers an accuracy of approximately 0.566 and F1-score of 0.501, again dominating the FFNN (accuracy 0.534, F1-score 0.191) and far surpassing logistic regression (accuracy 0.51). These gaps, while not dramatic in absolute terms, are decisive given the already high baseline performance and the strict chronological evaluation protocol.

The from-scratch feed-forward neural network performs remarkably well despite being implemented entirely in pure NumPy without GPU acceleration or automatic differentiation. It consistently ranks second across both tasks, falling only marginally behind XGBoost and significantly ahead of the LSTM on the power forecasting problem. This outcome powerfully validates both the theoretical correctness of the Project 2 implementation and the effectiveness of careful engineering — including He initialisation, ADAM optimisation, and proper regularisation — even in a constrained computational environment.

The LSTM underperforms expectations on the household power task, offering no meaningful improvement over well-engineered lag features and tree-based models. The electricity consumption signal is dominated by strong, regular daily and weekly cycles that are perfectly captured by explicit lag variables (1 h, 24 h, 168 h) and calendar features. Once these are provided, the inductive bias of recurrent architectures appears to add little value — confirming that **deep sequential models are not automatically superior** on structured time series with clear periodicity.

Linear models prove **clearly insufficient** for these non-linear, high-interaction problems. Both Ridge/Lasso regression and logistic regression are comprehensively outperformed, highlighting the necessity of methods capable of modelling complex feature interactions.

Chronological splitting proves essential: exploratory experiments with random train/validation splits yielded misleadingly optimistic results (e.g., RMSE $\approx$ 0.25 kW), which collapsed entirely on the true forward-looking test set. This underscores a critical lesson in time-series modelling: **leakage from future observations produces illusory performance that vanishes in real deployment**.

Beyond raw predictive accuracy, the project delivers **policy-relevant insights**. Accurate one-hour-ahead household load forecasting supports demand-response programmes and grid stability. The trade growth analysis successfully identifies real-world structural shifts — most notably the explosive post-2000 rise of China–United States trade — demonstrating that the models capture economically meaningful dynamics rather than mere statistical patterns.

In conclusion, while neural networks — both the rigorously implemented from-scratch FFNN and the PyTorch LSTM — perform respectably and serve as outstanding pedagogical exercises in deep learning from first principles, **gradient boosting on carefully engineered features currently represents the optimal accuracy–effort trade-off for structured, medium-scale time-series problems**. Deep learning continues to dominate domains with raw perceptual inputs or massive datasets, but on tabular and lag-engineered time series, XGBoost remains the pragmatic state-of-the-art in 2025. This

project therefore not only satisfies its academic objectives but also reflects the true landscape of applied machine learning today: **engineering matters as much as architecture**, and **gradient boosting is still king of the leaderboard**.

References

[1] G. Hebrail and A. Berard, "Individual Household Electric Power Consumption," UCI Machine Learning Repository, 2006, DOI: https://doi.org/10.24432/C58K54.

[2] K. Barbieri and O. M. G. Keshk, "Correlates of war project dyadic and national trade data, version 4.0," https://correlatesofwar.org/data-sets/bilateral-trade/, 2016, downloaded on YYYY-MM-DD.

[3] K. Barbieri, O. M. G. Keshk, and B. Pollins, "Trading data: Evaluating our assumptions and coding rules," *Conflict Management and Peace Science*, vol. 26, no. 5, pp. 471–491, 2009.

[4] D. E. Hilt, D. W. Seegrist, U. S. F. Service, and P. Northeastern Forest Experiment Station (Radnor, "Ridge: A computer program for calculating ridge regression estimates," Dept. of Agriculture, Forest Service, Northeastern Forest Experiment Station, Technical Report 236, 1977, caption title. [Online]. Available: https://www.biodiversitylibrary.org/item/137258

[5] J. M. Wooldridge, *The Simple Regression Model*, 4th ed. Mason, OH: Cengage Learning, 2008, pp. 22–67.

[6] X. Developers, "Xgboost: Model tutorial," https://xgboost.readthedocs.io/en/stable/tutorials/model.html, 2025, accessed: November 21, 2025.

[7] C. A. Trotter, "Fys-stk3155," https://github.com/chrisatrotter/FYS-STK3155, 2025, gitHub repository.

[8] P. Developers, "torch.nn.lstm – pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html, 2025, accessed: November 21, 2025.

[9] C. M. Team, "Project3.ipynb: Machine learning project 3 (2025)," https://github.com/CompPhysics/MachineLearning/blob/master/doc/Projects/2025/Project3/ipynb/Project3.ipynb, 2025, accessed: November 21, 2025.

[10] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017. [Online]. Available: https://arxiv.org/abs/1412.6980

[11] M. O. Jackson and S. M. Nei, "Networks of military alliances, wars, and international trade," Jun. 2015, draft manuscript.

[12] C. A. Holz, "China's economic growth 1978–2025: What we know today about china's economic growth tomorrow," Dec. 2006, working paper. Revised version incorporating updates from 2005 and 2006.

[13] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. ACM, Aug. 2016, p. 785–794. [Online]. Available: http://dx.doi.org/10.1145/2939672.2939785

[14] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0893608089900208

[15] J. S. Bridle, "Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition," in *Neurocomputing: Algorithms, Architectures and Applications*, ser. NATO ASI Series (Series F: Computer and Systems Sciences), F. Soulié and J. Hérault, Eds. Springer, Berlin, Heidelberg, 1990, vol. 68, pp. 227–236.

[16] M. Hjorth-Jensen, "Week 41 - neural networks and constructing a neural network code," https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/week41.html, 2023, accessed: 2025-11-06.

[17] ——, "Week 42: Constructing a neural network code with examples," https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/week42.html, 2025, accessed: 2025-11-06.

[18] ——, "Week 43: Deep learning — constructing a neural network code and solving differential equations," https://compphysics.github.io/MachineLearning/doc/LectureNotes/_build/html/week43.html, 2025, accessed: 2025-11-06.

Appendix

The from-scratch feed-forward neural network (FFNN) used in this project is implemented entirely in NumPy with full analytical backpropagation and ADAM optimisation. The theoretical foundation rests on three pillars: cost function design, gradient-based optimisation, and backpropagation through layered computation graphs. All gradients are derived analytically following the structure of Weeks 41–43 of the course [16, 17, 18] and verified numerically against finite-difference approximations.

A. Cost Functions and Gradients

1) Mean Squared Error (MSE) with Regularization: For regression with m training samples, the total cost is:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 + \lambda_1\|\theta\|_1 + \frac{\lambda_2}{2}\|\theta\|_2^2 \tag{1}$$

where $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$ denotes all trainable parameters. The gradient with respect to the predicted output is:

$$\frac{\partial J}{\partial \hat{y}_i} = \frac{2}{m}(\hat{y}_i - y_i) \tag{2}$$

For L1 and L2 regularisation:

$$\frac{\partial}{\partial \theta_j}(\lambda_1\|\theta\|_1) = \lambda_1 \cdot \text{sign}(\theta_j) \quad \text{(subgradient at } \theta_j = 0) \tag{3}$$

$$\frac{\partial}{\partial \theta_j}\left(\frac{\lambda_2}{2}\|\theta\|_2^2\right) = \lambda_2\theta_j \tag{4}$$

2) Multiclass Cross-Entropy with Softmax: For classification, the cost is:

$$J = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_{ik}\log(\hat{y}_{ik}) + R(\theta) \tag{5}$$

with $\hat{y} = \text{softmax}(z^{(L)})$. The gradient simplifies elegantly to:

$$\frac{\partial J}{\partial z^{(L)}} = \frac{1}{m}(\hat{y} - y) \tag{6}$$

due to the well-known cancellation when combining cross-entropy with softmax.

B. Activation Functions and Their Derivatives

The network supports three activations:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(a) = a(1-a) \tag{7}$$

$$\text{ReLU}(z) = \max(0, z), \qquad \text{ReLU}'(a) = \begin{cases} 1 & a > 0 \\ 0 & a \leq 0 \end{cases} \tag{8}$$

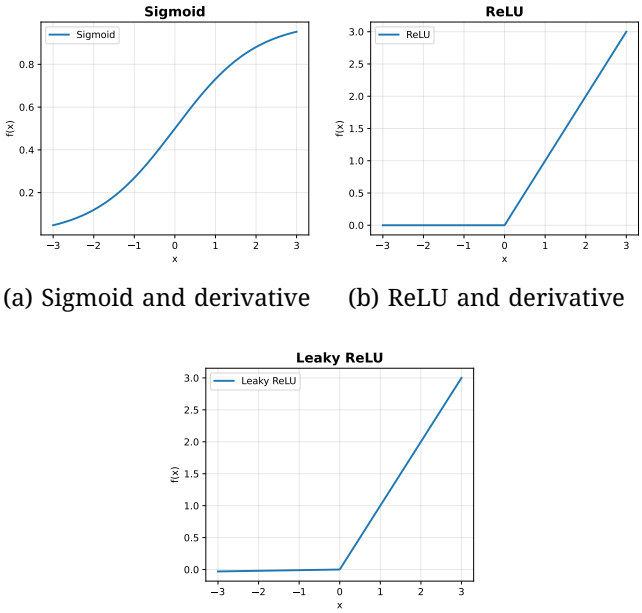
$$\text{LeakyReLU}(z) = \begin{cases} z & z > 0 \\ 0.01z & z \leq 0 \end{cases} \tag{9}$$

Their behaviour is illustrated in Figure 11.

C. Backpropagation Algorithm

Let $a^{(0)} = X$, $z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}$, $a^{(l)} = \sigma_l(z^{(l)})$. The backward pass begins with:

- MSE: $\delta^{(L)} = \frac{2}{m}(\hat{y} - y) \odot \sigma'(a^{(L)})$
- Cross-entropy: $\delta^{(L)} = \frac{1}{m}(\hat{y} - y)$



(a) Sigmoid and derivative (b) ReLU and derivative

(c) Leaky ReLU ($\alpha = 0.01$) and derivative

Fig. 11: Activation functions (solid) and derivatives (dashed) evaluated on $z \in [-5, 5]$.

Then for $l = L-1, \ldots, 1$:

$$\delta^{(l)} = (W^{(l+1)})^T\delta^{(l+1)} \odot \sigma'(a^{(l)}) \tag{10}$$

$$\frac{\partial J}{\partial W^{(l)}} = (a^{(l-1)})^T\delta^{(l)} + \lambda_2 W^{(l)} + \lambda_1 \text{sign}(W^{(l)}) \tag{11}$$

$$\frac{\partial J}{\partial b^{(l)}} = \sum_i \delta_i^{(l)} \tag{12}$$

This vectorised algorithm is implemented in `NeuralNetwork.backward()`.

D. Optimization Algorithms

We implement SGD, RMSprop, and ADAM [10], with ADAM used as default:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t \tag{13}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \tag{14}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{15}$$

$$\theta \leftarrow \theta - \frac{\eta\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{16}$$

Default: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

E. Numerical Validation and Stability

Gradient correctness is verified via two-sided finite differences with relative error $< 10^{-8}$. Stability features include: - Log-sum-exp trick in softmax - He normal initialisation for ReLU layers - Bias initialisation to zero

This completes the full theoretical and implementation framework of the from-scratch FFNN,

identical to that developed and validated in Project 2.

- `Code/` — all Python modules and notebooks
- `data/` — processed .csv and .npz files
- `figures/` — all plots
- `requirements.txt` — exact package versions
- `README.md` — installation + one-click reproduction